

7.6 研究日志整理版

light_cci + Laplacian-HKS + Sparse Semi-relaxed OT

目标：用低成本 CCI/DDI 图结构产生跨时间 PIJ，并避免大规模 expression/CCI mix 特征导致的计算爆炸。

核心边界。本方案的主结果只使用每个层级自身的 light_cci 图、Laplacian-HKS 特征、SR/发育状态量、空间位置，以及跨时间候选边。主 PIJ 不使用 lower-to-upper overlap、domain 映射或跨层标签作为成本或候选边来源，避免把 upper 层结构提前泄露给 lower 层。

Contents

1 总体思路	3
2 符号系统	3
2.1 样本组与节点集合	3
2.2 gene 层级：unit 是 gene，边是 GRN	3
2.3 LR pair、COMMOT 矩阵与现成 CCI total	4
3 Laplacian-HKS 节点特征	5
3.1 从 light_cci 邻接矩阵到 normalized Laplacian	5
3.2 谱分解与 HKS	5
4 source-target 预成本与候选集合	6
4.1 两个时间点的图与节点特征	6
4.2 预成本 B_{ij}	6
4.3 候选集合 Ω	6
5 稀疏候选成本：暂不使用辅助图内部边	7
5.1 最终 OT 成本	7
5.2 为什么先去掉辅助图内部边	7
6 Sparse Semi-relaxed Entropic OT	7
6.1 目标函数	7
6.2 为什么只保留 source 侧 KL	8
6.3 推荐优化方法	8
7 代码落地补充操作	9
7.1 代码层次必须拆清楚	9
7.2 新增 network method: light_cci	9
7.3 新增 HKS 表征函数	10
7.4 新增 PIJ method: laplacian_sr_sparse_semirelaxed_ot	11
7.5 配置项建议	11
7.6 profile-only 必须补	11
7.7 输出文件建议	12

8 实现时需要避免的坑	12
9 最小验证实验	12
10 建议命令形态	13
10.1 先 profile-only	13
10.2 再正式跑	13
11 结论	13

1 总体思路

现在的问题不是“有没有 OT”，而是原来的表达特征太大，且完整 source-target 成本矩阵的规模是二次的。新的路线把任务拆成四步：

1. **先建轻量图**。每个 organ、时间点、层级只构造 light_cci 图：cell/spot 层用 CCI，domain 层用 DDI 或聚合后的 CCI/DDI；新增 gene 层时，节点不再是 cell/domain，而是 gene，边来自 group 内的 gene×gene GRN 矩阵。这里的 gene 层不是 GGI，也不是 LR 通讯，而是基因调控网络。
2. **再做图结构表征**。对每个时间点内部图做 Laplacian-HKS，把每个节点压缩为低维结构特征。
3. **用预成本筛候选**。用 Laplacian-HKS、SR/发育状态量、空间坐标构造预成本 B_{ij} ，只保留可能的跨时间候选边 Ω 。
4. **在稀疏候选上做半松弛 OT**。直接在候选边上定义最终成本 C_{ij} ，再用 sparse semi-relaxed entropic OT 优化得到 P_{ij} 。

light_cci 图 $\rightarrow$ Laplacian-HKS 特征 $\rightarrow$ KNN 候选边 $\rightarrow$ sparse semi-relaxed OT $\rightarrow P_{ij}$ 。

本版替换点。旧版里辅助图会把两个时间点内部的 CCI 边 E_0, E_1 再放进路径成本。当前版本先不使用这部分，只保留候选出来的跨时间边 Ω ，即 C_{ij} 直接来自候选边的预成本。这样模型更轻，符号更简单，也避免 CCI 信息在 HKS 和辅助图中双重参与。

2 符号系统

2.1 样本组与节点集合

$$g = (c', t, \ell).$$

这里， g 表示一个样本组； c' 表示器官或组织，例如 heart； t 表示发育时间点； ℓ 表示层级，例如 cell、spot、domain 或 gene-level layer。

对任意 $g = (c', t, \ell)$ ，定义：

$$V_{\ell, t}^{(c')} = \{\text{该器官、该时间点、该层级的所有节点}\}.$$

这里， $V_{\ell, t}^{(c')}$ 是节点集合；如果 ℓ 是 cell/spot 层，节点就是 cell/spot；如果 ℓ 是 domain 层，节点就是 domain；如果 ℓ 是基因层，节点就是跨细胞 GRN 中的 gene。

2.2 gene 层级：unit 是 gene，边是 GRN

为了保留 spot 级别已有的大 GRN 信息，可以在 light_cci 家族中新增一个 gene 层。这个层级和 CCI/DDI 层的含义不同：**节点不是 cell、spot 或 domain，而是 gene；gene 之间的边也不是 GGI，更不是 LR 通讯，而是 GRN 调控边。**

对一个 group $g = (c', t, \ell = \text{gene})$ ，定义 gene 节点集合：

$$\mathcal{G}_{c', t} = \{\text{该器官、该时间点用于 GRN 的基因}\}.$$

这里， $\mathcal{G}_{c', t}$ 是 gene 层节点集合。每个 group 已有一个 gene×gene 的 GRN 矩阵：

$$R_{c',t}^{\text{GRN}} \in \mathbb{R}^{|\mathcal{G}_{c',t}| \times |\mathcal{G}_{c',t}|}.$$

这里, $R_{c',t}^{\text{GRN}}$ 是该 group 的基因调控矩阵; 元素 R_{ab}^{GRN} 表示调控基因 a 对目标基因 b 的调控强度或调控置信度。

第一版 gene 层图可以写成:

$$G_{\text{gene},t}^{(c')} = (\mathcal{G}_{c',t}, E_{\text{GRN}}, w_{\text{GRN}}).$$

其中, E_{GRN} 是从 $R_{c',t}^{\text{GRN}}$ 的非零项或 top 边过滤后得到的有向调控边集合; $w_{\text{GRN}}(a, b)$ 是 gene $a \rightarrow b$ 的调控边权。

gene 层和其他层的边类型不同。 cell/spot 层的边来自 CCI, domain 层的边来自 DDI 或聚合 CCI/DDI, gene 层的边来自 GRN。因此 light_cci 在代码里应理解为“轻量层级网络构造器”, 而不是所有层都强行使用 CCI。

如果 GRN 是有符号矩阵, Laplacian-HKS 不能直接使用负边权。第一版建议把拓扑强度定义为:

$$A_{\text{gene},t}^{\text{GRN}}(a, b) = |R_{ab}^{\text{GRN}}|.$$

这里, $A_{\text{gene},t}^{\text{GRN}}$ 是进入 Laplacian-HKS 的非负 gene-level 邻接矩阵。符号信息可以作为 edge attribute 另存, 后续再做激活/抑制方向解释; 第一版 PIJ 先只使用调控强度, 不把正负号直接放进 Laplacian。

2.3 LR pair、COMMOT 矩阵与现成 CCI total

对某一个 ligand-receptor pair q , COMMOT 可以给出一个稀疏通讯矩阵:

$$M_{\ell,t}^{(q)} \in \mathbb{R}_+^{|\mathcal{V}_{\ell,t}^{(c')}| \times |\mathcal{V}_{\ell,t}^{(c')}|}.$$

这里, $M_{\ell,t}^{(q)}$ 表示在层级 ℓ 、时间 t 上, 第 q 个 LR pair 的通讯矩阵; 矩阵元素 $M_{uv}^{(q)}$ 表示节点 u 通过该 LR pair 对节点 v 的通讯强度。

从数学上, 所有 LR pair 可以聚合成一张 light_cci 邻接矩阵:

$$A_{\ell,t} = \sum_{q \in Q_{\ell,t}} M_{\ell,t}^{(q)}.$$

这里, $Q_{\ell,t}$ 表示该层级、该时间点可用的 LR pair 集合; $A_{\ell,t}$ 是聚合后的有向通讯强度矩阵。

工程实现优先级。 如果输入目录里已经有 *_CCI_total.npz, 第一版 light_cci 应该直接读取这个 total 矩阵作为 $A_{\ell,t}$, 再用 *_index.tsv 对齐节点顺序。只有在 total 矩阵不存在时, 才回退到逐个读取 COMMOT_by_LR 里的 $M^{(q)}$ 并求和。

这和上面的数学聚合思路是一致的: CCI_total.npz 可以视为 pipeline 已经替我们完成的 LR 聚合结果。代价是暂时不保留“这条边来自哪个 LR pair”的解释细节; 当前轻量 PIJ 主线可以接受这个损失。

3 Laplacian-HKS 节点特征

3.1 从 light_cci 邻接矩阵到 normalized Laplacian

普通 graph Laplacian 通常需要无向图。因此先把有向通讯矩阵对称化：

$$\tilde{A}_{\ell,t} = \frac{1}{2} (A_{\ell,t} + A_{\ell,t}^\top).$$

这里， $\tilde{A}_{\ell,t}$ 是对称后的邻接矩阵； $A_{\ell,t}^\top$ 表示 $A_{\ell,t}$ 的转置。
定义度矩阵：

$$D_{\ell,t} = \text{diag}(\tilde{A}_{\ell,t} \mathbf{1}).$$

这里， $D_{\ell,t}$ 是对角矩阵； $\mathbf{1}$ 是全 1 向量； $\tilde{A}_{\ell,t} \mathbf{1}$ 得到每个节点的加权重。
normalized Laplacian 定义为：

$$L_{\ell,t} = I - D_{\ell,t}^{-1/2} \tilde{A}_{\ell,t} D_{\ell,t}^{-1/2}.$$

这里， $L_{\ell,t}$ 是 normalized Laplacian； I 是单位矩阵； $D_{\ell,t}^{-1/2}$ 表示对角元素开方后取倒数。

3.2 谱分解与 HKS

对 $L_{\ell,t}$ 做特征分解：

$$L_{\ell,t} \phi_m = \lambda_m \phi_m, \quad m = 1, 2, \dots, K.$$

这里， m 是第几个谱分量的编号； K 是保留的谱分量数量； λ_m 是第 m 个特征值； ϕ_m 是对应的特征向量； $\phi_m(i)$ 表示该特征向量在节点 i 上的取值。

直接使用 eigenvector 坐标会有符号翻转和旋转不唯一问题。因此主方案使用 heat kernel signature：

$$z_{i,h}^{\text{lap}} = \sum_{m=1}^K \exp(-\tau_h \lambda_m) \phi_m(i)^2, \quad h = 1, 2, \dots, H.$$

这里， $z_{i,h}^{\text{lap}}$ 是节点 i 在第 h 个扩散尺度上的 HKS 值； h 是扩散尺度编号； H 是扩散尺度数量； τ_h 是第 h 个 heat diffusion time； $\phi_m(i)^2$ 用平方消除 eigenvector 的符号任意性。

最终节点特征为：

$$z_i^{\text{lap}} = [z_{i,1}^{\text{lap}}, z_{i,2}^{\text{lap}}, \dots, z_{i,H}^{\text{lap}}].$$

默认超参建议。第一版可以设 $K = 32$, $H = 6$ 。 τ_h 建议按 Laplacian 特征值范围自动取 log-spaced scales, 例如从 $1/(\lambda_K + \epsilon)$ 到 $1/(\lambda_2 + \epsilon)$ 取 H 个对数间隔点。

4 source-target 预成本与候选集合

4.1 两个时间点的图与节点特征

对一个时间对 (t_0, t_1) , 定义 source 图和 target 图:

$$G_0 = (S, E_0, w_0), \quad G_1 = (T, E_1, w_1).$$

这里, G_0 是 source 时间 t_0 的 light_cci 图; G_1 是 target 时间 t_1 的 light_cci 图; S 是 source 节点集合; T 是 target 节点集合; E_0 和 E_1 分别是两个时间点内部的 CCI/DDI 边集合; w_0 和 w_1 是内部边权函数。

本版边界。 E_0, E_1 只用于从 G_0, G_1 构造 Laplacian-HKS 特征。当前主 PIJ 不再把 E_0, E_1 放入辅助图, 也不再用内部 CCI 边计算最短路成本。

对 $i \in S$ 和 $j \in T$, 定义:

- z_i : source 节点 i 的 Laplacian-HKS 特征; z'_j : target 节点 j 的 Laplacian-HKS 特征。
- q_i : source 节点 i 的 SR 或发育状态量; q'_j : target 节点 j 的 SR 或发育状态量。
- r_i : source 节点 i 的空间坐标; r'_j : target 节点 j 的空间坐标。这里使用 r , 避免和表达矩阵 X 混淆。
- a_i : source 节点 i 的给定质量; b_j : target 节点 j 的给定质量。

4.2 预成本 B_{ij}

候选筛选只需要一个快速预成本, 不需要一开始就计算完整 dense 成本矩阵。对于 cell/spot/domain 层, 可以同时使用 HKS、SR/发育状态和空间坐标; 对于 gene 层, 节点是 gene, 通常没有空间坐标, 因此默认令 $\lambda_r = 0$ 。如果没有可靠的 gene-level SR 或发育状态量, 也可以令 $\lambda_q = 0$, 只用 GRN-HKS 特征筛选。

$$B_{ij} = \frac{\lambda_z d_z(z_i, z'_j) + \lambda_q d_q(q_i, q'_j) + \lambda_r d_r(r_i, r'_j)}{\lambda_z + \lambda_q + \lambda_r}.$$

这里, B_{ij} 是从 source 节点 i 到 target 节点 j 的预成本; d_z 是 HKS 特征距离; d_q 是 SR/发育状态距离; d_r 是空间距离; $\lambda_z, \lambda_q, \lambda_r$ 是三个距离分量的非负权重。

重要解释。 B_{ij} 越小, 说明 i 和 j 越相似, 越应该优先保留。 B_{ij} 既用于筛候选边, 也作为当前主方案的最终 OT 成本来源; 不再额外构造含内部 CCI 边的辅助图成本。

4.3 候选集合 Ω

对每个 source 节点 i , 保留 B_{ij} 最小的 K_s 个 target 节点; 对每个 target 节点 j , 保留 B_{ij} 最小的 K_t 个 source 节点。候选集合写成:

$$\begin{aligned} \Omega &= \Omega_{S \rightarrow T} \cup \Omega_{T \leftarrow S}. \\ \Omega_{S \rightarrow T} &= \{(i, j) : j \in \text{TopK}_{K_s}^{\text{small}}(B_{i,:})\}, \\ \Omega_{T \leftarrow S} &= \{(i, j) : i \in \text{TopK}_{K_t}^{\text{small}}(B_{:,j})\}. \end{aligned}$$

这里， Ω 是跨时间候选边集合； K_s 是每个 source 保留多少个 target 候选； K_t 是每个 target 反向保留多少个 source 候选。

覆盖诊断必须输出。 每个 source 是否至少有 K_s 个候选；每个 target 是否至少有若干候选流入；候选边覆盖了多少连通分量； $K_s = 20, 50, 100$ 时 EI/DI 是否稳定。这些诊断比单独报告一个最终 EI 更重要。

5 稀疏候选成本：暂不使用辅助图内部边

5.1 最终 OT 成本

当前主方案直接在候选集合上定义成本：

$$C_{ij} = \text{normalize}_{(i,j) \in \Omega}(B_{ij}), \quad (i, j) \in \Omega.$$

这里， C_{ij} 是最终进入 OT 的成本；`normalize` 表示只在候选边集合 Ω 上做尺度归一化，例如分位数截断后缩放到 $[0, 1]$ 或做 `robust z-score` 后再缩放。

本版替换。 不再构造 $H = (S \cup T, E_0 \cup E_1 \cup \Omega, w_H)$ ，不再计算 $\text{dist}_H(i, j)$ ，不再使用两个时间点内部 CCI 边作为路径成本。这样做的目的不是否认辅助图，而是第一版先把模型压到最轻、最清楚、最容易落地的形态。

5.2 为什么先去掉辅助图内部边

Laplacian-HKS 已经从 `light_cci` 图中提取了节点结构特征。如果再把 E_0, E_1 放进辅助图并用 CCI 最短路生成 C_{ij} ，CCI 信息会在“节点表征”和“路径成本”两个阶段都参与。它们功能不同，但可能导致 CCI 过度放大，尤其在存在 hub 或弱边过多时。

当前版本采用更克制的建模：

$$\text{CCI 图} \rightarrow \text{HKS 节点特征} \rightarrow B_{ij} \rightarrow \Omega \rightarrow C_{ij} \rightarrow P_{ij}.$$

这意味着 CCI 主要通过 HKS 进入跨时间成本，SR/空间项作为补充。后续如果主方案跑通且需要增强，再单独设计辅助图版本；但当前 PDF 和第一版代码落地不包含辅助图内部 CCI 边。

6 Sparse Semi-relaxed Entropic OT

6.1 目标函数

在稀疏候选集合上求转移矩阵：

$$P \in \mathbb{R}_+^{|S| \times |T|}, \quad \text{supp}(P) \subseteq \Omega.$$

这里， P_{ij} 表示从 source 节点 i 到 target 节点 j 的转移强度； $\text{supp}(P)$ 表示 P 的非零位置集合。

主优化目标改成 semi-relaxed OT：

$$\min_{P \geq 0, \text{supp}(P) \subseteq \Omega} \sum_{(i,j) \in \Omega} P_{ij} C_{ij} + \varepsilon \sum_{(i,j) \in \Omega} P_{ij} (\log P_{ij} - 1) + \gamma \text{KL}(P\mathbf{1} \| a),$$

$$\text{s.t.} \quad P^\top \mathbf{1} = b.$$

这里，第一项让大的 P_{ij} 尽量落在小的 C_{ij} 上；第二项是 **entropy regularization**，使解更平滑、更可优化； ε 控制平滑强度；第三项是 **source** 侧质量守恒的软约束； γ 控制 **source** 质量守恒强度；**target** 侧使用硬约束 $P^\top \mathbf{1} = b$ ，表示 **target** 时间点的观测状态都需要被解释。

KL 项定义为：

$$\text{KL}(r \| a) = \sum_i \left[r_i \log \frac{r_i}{a_i} - r_i + a_i \right], \quad r = P\mathbf{1}.$$

这里， $r_i = \sum_j P_{ij}$ 是 **source** 节点 i 实际送出的质量； a_i 是 **source** 节点 i 的给定质量。

6.2 为什么只保留 **source** 侧 KL

当前任务是发育时间对 (t_0, t_1) 。**target** 时间点 t_1 已经被观测到，所以每个 **target** 节点都应该从前一时间点获得解释：

$$P^\top \mathbf{1} = b.$$

source 时间点 t_0 的节点则可以有多多个后代、少量后代，甚至没有清晰后代。因此 **source** 侧使用软约束：

$$P\mathbf{1} \approx a.$$

如果某个 **source** 节点 i 没有合理 **target**，模型可以让：

$$r_i = \sum_j P_{ij} < a_i.$$

这表示它没有把全部质量送出，数学上对应“状态减少、消失或没有清晰后继”。偏离越大，KL 惩罚越大，因此模型不是任意丢质量，而是在“强行匹配的路径成本”和“**source** 质量不守恒惩罚”之间做权衡。

和 fully unbalanced 的区别。旧版使用 **source** 和 **target** 两侧 KL，形式更灵活，但解释更散。当前 **semi-relaxed** 版本只放松 **source** 侧，**target** 侧固定。这更符合“后一个时间点每个观测状态都要被解释，前一个时间点状态可以扩增或消失”的发育叙事，也更容易说服导师。

6.3 推荐优化方法

目标函数使用 **sparse semi-relaxed Sinkhorn** 优化。先定义稀疏 Gibbs kernel：

$$K_{ij} = \exp \left(-\frac{C_{ij}}{\varepsilon} \right), \quad (i, j) \in \Omega.$$

只在候选边上存储 K_{ij} 。semi-relaxed Sinkhorn 可以理解为：target 侧缩放要严格满足 b ，source 侧缩放按 KL 强度做软更新。工程上应该在 log-domain 中实现，避免数值下溢。一个可实现的更新形式是：

$$\begin{aligned}\rho &= \frac{\gamma}{\gamma + \varepsilon}, \\ u &\leftarrow \left(\frac{a}{Kv + \delta} \right)^\rho, \quad v \leftarrow \frac{b}{K^\top u + \delta}. \\ P_{ij} &= u_i K_{ij} v_j, \quad (i, j) \in \Omega.\end{aligned}$$

这里， u 和 v 是 Sinkhorn scaling vectors； K 是稀疏 kernel； δ 是数值稳定项； ρ 是 source 侧 semi-relaxed 更新指数。由于 target 侧是硬约束， v 的更新不带 unbalanced 指数。

为了兼容当前 EI 代码，如果后续需要行随机 PIJ，可以导出：

$$\hat{P}_{ij} = \frac{P_{ij}}{\sum_{j'} P_{ij'} + \delta}.$$

同时必须保留未归一化的 P ，用于 source 质量减少/扩增诊断。

7 代码落地补充操作

7.1 代码层次必须拆清楚

当前代码修改应该明确分成三层：

1. **network method**: light_cci。只负责读取/构造每个层级、每个时间点的轻量 CCI/DDI 图，输出 LayerGraph 和必要 metadata。
2. **representation**: Laplacian-HKS。它不是 network method，而是 PIJ 方法内部调用的图表征步骤；输入是 light_cci 输出的图，输出是节点 HKS 特征。
3. **PIJ method**: laplacian_sr_sparse_semirelaxed_ot。负责 HKS、SR/空间预成本、KNN 候选、稀疏成本、semi-relaxed Sinkhorn，以及 PIJ 导出。

必须备注到代码计划里。不要让 light_cci 直接输出 Laplacian 后的特征。light_cci 的职责是 network builder；第二层 HKS 和第三层 OT 都属于 PIJ method 的内部流程或可复用工具函数。

7.2 新增 network method: light_cci

新增文件：

```
mignet_ce/networks/light_cci.py
```

建议直接复用当前 clean_expression_cci_mix 的路径检查和 unit/index 对齐逻辑，但不要构建大 expression block，也不要构建 native CCI feature matrix。

对 cell/spot/domain 层，优先读取：

```
*_CCI_total.npz
*_index.tsv
```

对 gene 层，读取 group 内已有的 gene×gene GRN 矩阵，例如：

```
*_GRN_total.npz # 或项目中已有的 gene x gene GRN 文件名
*_gene_index.tsv # 记录矩阵行列对应的 gene
```

如果 CCI total 矩阵不存在, 再回退到:

```
*_COMMOT_lr_pairs.tsv
*_COMMOT_by_LR/*.npz
```

输出的 NetworkContext 中, light_cci 需要按层级分派数据源:

- $\ell = \text{cell}$ 或 $\ell = \text{spot}$: 读取 CCI total 图;
- $\ell = \text{domain}$: 读取 DDI 或由 CCI 聚合得到的 domain-level 图;
- $\ell = \text{gene}$: 读取 gene $\times$ gene GRN 图, unit 顺序由 gene index 文件决定;
- gene 层的 GRN 不注入 cell/spot/domain 层, 避免把 spot 级大 GRN 再混入 CCI 层特征。

输出的 NetworkContext 中:

- lower_graphs 和 upper_graphs 保留 light_cci 图;
- lower_mats 和 upper_mats 可以先放 np.zeros((n_units, 0)) 占位;
- feature_alignment_space="native_units";
- metadata["feature_source"]="light_cci_graph_only";
- metadata["uses_grn"] 对 cell/spot/domain 层为 False, 对 gene 层为 True;
- metadata["grn_scope"]="gene_layer_only";
- metadata["uses_cci"]=True;
- metadata["uses_expression_feature"]=False;
- 表达数据只允许用于读取节点、坐标、可选 LR 表达过滤, 不再作为高维特征进入 PIJ。

需要修改:

```
mignet_ce/networks/registry.py
mignet_ce/config.py
```

把 light_cci 加入 NETWORK_BUILDERS 和 NETWORK_METHODS。

7.3 新增 HKS 表征函数

当前代码已有普通 Laplacian embedding。需要新增 HKS, 而不是覆盖旧方法:

```
mignet_ce/embeddings.py
- graph_to_adjacency(...)
- laplacian_hks_features(...)
- layer_graph_laplacian_hks_features(...)
```

关键要求:

- 支持 n_eigen=32、n_scales=6;
- 支持 tau_strategy="auto";
- 对孤立点返回零向量或小常数向量;
- 输出每个节点的 H 维 HKS 特征;
- 导出 eigen convergence、实际保留特征数、耗时和内存诊断。

7.4 新增 PIJ method: laplacian_sr_sparse_semirelaxed_ot

新增文件：

```
mignet_ce/pij/laplacian_sr_sparse_semirelaxed_ot.py
mignet_ce/transition/sparse_semirelaxed_ot.py
```

主流程：

1. 从 `context.lower_graphs[t]` 和 `context.upper_graphs[t]` 计算 HKS 特征。
2. 读取 development feature table 中的 SR 或发育状态量。
3. 从 `context.lower_coords_by_time` 和 `context.upper_coords_by_time` 读取坐标。
4. 构造标准化预成本 B_{ij} ，用 approximate nearest neighbors 选候选 Ω 。
5. 直接令 $C_{ij} = \text{normalize}(B_{ij})$ ，只在 Ω 上保存稀疏成本。
6. 在 sparse C 上运行 semi-relaxed Sinkhorn。
7. 导出 raw sparse PIJ、行归一化 PIJ、候选覆盖诊断和 source mass 诊断。

7.5 配置项建议

在 `TemporalRunConfig` 中增加：

```
hks_eigen_components: int = 32
hks_scales: int = 6
hks_tau_strategy: str = "auto"
sparse_candidate_k_source: int = 50
sparse_candidate_k_target: int = 20
sparse_use_reverse_knn: bool = True
cost_weight_hks: float = 1.0
cost_weight_sr: float = 1.0
cost_weight_spatial: float = 1.0
sparse_ot_epsilon: float = 0.05
sparse_ot_source_kl_gamma: float = 1.0
sparse_ot_max_iter: int = 200
sparse_ot_tol: float = 1e-6
profile_only: bool = False
```

旧版与辅助图相关的配置项暂不加入第一版，例如：

```
aux_internal_topk_per_node
aux_gamma_source
aux_gamma_target
dijkstra_batch_size
```

7.6 profile-only 必须补

新增脚本或运行模式：

```
scripts/profile_light_cci_pij.py
# 或者在 run_mignet_vertical_ablation.py 里加 --profile-only
```

profile-only 不真正跑完整 PIJ，只输出规模估计：

- 每个 organ/time/layer 的节点数；
- 是否存在 *_CCI_total.npz、矩阵 shape、nnz、文件大小；
- gene 层是否存在 gene×gene GRN 矩阵、gene 数、GRN nnz、过滤后边数；

- 如果需要 fallback, 才统计 LR manifest 文件数、LR 文件总大小、总 nnz、最大 nnz;
- 预计 CCI graph edge 数量;
- Laplacian adjacency nnz、预计 eigensolver 规模;
- 候选边 $|\Omega| \approx |S|K_s + |T|K_t$;
- sparse OT memory: $\text{nnz} * (\text{cost} + \text{row} + \text{col} + \text{kernel} + P)$;
- dense OT memory 对照: $|S| \times |T| \times 8 \times \text{multiplier}$;
- 预估能否上 GPU, 或是否应该 CPU sparse。

7.7 输出文件建议

每个 organ、层级对、时间对建议输出:

```
profile.json
units_source.csv
units_target.csv
hks_source.npy
hks_target.npy
candidate_edges.parquet
cost_sparse.npz
pij_transport_sparse.npz
pij_row_normalized_sparse.npz
source_mass_diagnostics.csv
ot_convergence.json
```

其中 `source_mass_diagnostics.csv` 至少包含 source 实际送出质量 r_i 、原质量 a_i 、比值 r_i/a_i , 用于解释哪些 source 状态可能减少、扩增或没有清晰后继。

8 实现时需要避免的坑

1. 不要生成完整 $|S| \times |T|$ 成本矩阵。候选筛选、成本、OT 都应该保持稀疏。
2. 不要把 overlap/domain 映射用于主 PIJ。profile 可以检查 coverage, 但不能把跨层映射放进 lower 层 PIJ 成本。
3. 不要让 light_cci 输出 HKS 特征。network method 只输出图, HKS 和 OT 属于 PIJ method。
4. 不要第一版就加入辅助图内部 CCI 边。当前成本只使用候选边上的 $C_{ij} = \text{normalize}(B_{ij})$ 。
5. 保留 raw transport。行归一化 PIJ 可以给 EI 使用, 但 raw P 对解释 source mass 非常重要。
6. 如果使用 CCI_total.npz, 必须用 index 文件确认矩阵行列顺序和 unit 顺序一致。
7. gene 层必须用 gene index 对齐 GRN 矩阵行列; 不要把 gene×gene GRN 当成 cell×cell CCI, 也不要把它命名为 GGI。
8. gene 层默认没有空间坐标, 预成本中应设置 $\lambda_r = 0$; 没有 gene-level SR 时设置 $\lambda_q = 0$ 。

9 最小验证实验

实验	目的
profile-only K sensitivity	在真正跑之前估算 total CCI nnz、候选边、稀疏 OT 内存。 比较 $K_s = 20, 50, 100$ 下 EI/DI 是否稳定, 防止 KNN 删掉真实对应。

source KL sensitivity semi-relaxed vs balanced randomized CCI control	比较 $\gamma = 0.1, 1, 10$, 检查 source 质量松弛是否显著影响 EI 结论。 把 γ 设得很大近似 balanced OT, 比较半松弛是否真的改变结果。 打乱 CCI 边权或保持度分布随机化, 确认结果不是由节点数或边密度本身驱动。
--	--

10 建议命令形态

10.1 先 profile-only

```
python scripts/profile_light_cci_pij.py \
--data-root public/datasets/Mouse-embryo/E1S1_domain_factory \
--output-root output/profile_light_cci \
--organs heart \
--time-points 11.5 12.5 13.5 14.5 \
--network-method light_cci \
--pij-method laplacian_sr_sparse_semirelaxed_ot \
--candidate-k-source 50 \
--candidate-k-target 20 \
--profile-only \
--progress
```

10.2 再正式跑

```
python scripts/run_mignet_vertical_ablation.py \
--data-root public/datasets/Mouse-embryo/E1S1_domain_factory \
--output-root output/light_cci_laplacian_sr_sparse_semirelaxed_ot \
--organs heart \
--time-points 11.5 12.5 13.5 14.5 \
--network-method light_cci \
--pij-method laplacian_sr_sparse_semirelaxed_ot \
--development-feature-root public/datasets/Mouse-embryo/E1S1_domain_factory/
development_features \
--export-pij \
--progress \
--progress-log output/light_cci_laplacian_sr_sparse_semirelaxed_ot/progress.
jsonl
```

11 结论

这个版本的重点不是把所有信息都堆进成本函数, 而是把每一种信息放到合适的位置: **light_cci** 图提供低成本网络骨架, 其中 cell/spot/domain 层使用 CCI/DDI, gene 层使用 gene×gene GRN; Laplacian-HKS 提供稳定节点结构表征; SR/空间信息辅助筛候选; 候选边成本直接作为稀疏 OT 成本; sparse semi-relaxed OT 在稀疏候选上优化 PIJ, 并允许 source 状态减少、扩增或没有清晰后继。

主方案: light_cci + Laplacian-HKS candidate + sparse semi-relaxed Sinkhorn.

LightCCI 对比实验矩阵修订增补

feature axis $\times$ PIJ method axis

本增补只追加新的对比实验定义与代码落地边界；前文原有内容不在此处重写。

1 对比实验矩阵的定位

本轮新增实验的目标，是把 LightCCI 网络下的 PIJ 构造拆成一个二维消融矩阵：横轴是节点特征如何构造，纵轴是如何把节点特征转成跨时间 PIJ。这里的 LightCCI 首先提供层级节点集合、节点顺序、层级内部图、坐标和必要 metadata；而具体某一列 feature 是否使用 CCI 边，由该列特征定义决定。

因此，LightCCI 是本组实验共同的 network method。新增的一组 PIJ 方法统一放在 `mignet_ce/pij/` 目录下，并且新增文件统一使用 `compare_` 作为文件名前缀。为了便于单独运行、单独调参、单独统计时间和单独定位问题，本组矩阵不使用一个大文件通过参数切换特征或 PIJ 方式，而是要求矩阵中的每一个格子对应一个独立 PIJ 方法文件。

1.1 新增 PIJ 文件命名要求：一个矩阵格子一个文件

设特征列为

$$E, N, L, Sr, E_N, E_L, E_Sr, N_L, N_Sr, L_Sr,$$

这里， E 表示 expression feature； N 表示 node-factor feature； L 表示 Laplacian-HKS feature； Sr 表示 SR/developmental feature；下划线表示两类基础特征的组合。

设 PIJ 生成方法为

$$cos, \quad kl, \quad sot.$$

这里， cos 表示 cosine kernel 到 PIJ； kl 表示 feature-level KL 到 PIJ； sot 表示 sparse semi-relaxed OT 到 PIJ。

则矩阵中的每个格子都应落成一个文件：

$$\text{mignet_ce/pij/compare_}<\text{feature}>_<\text{method}>.\text{py}.$$

这里，`<feature>` 是当前格子的特征组合名；`<method>` 是当前格子的 PIJ 生成方法名。这个文件就是一个可以注册到 `registry.py` 的具体 PIJ method，而不是只在一个公共文件里传入不同参数。

特征列	对应三个 PIJ 文件
E	<code>compare_E_cos.py</code> ; <code>compare_E_kl.py</code> ; <code>compare_E_sot.py</code>
N	<code>compare_N_cos.py</code> ; <code>compare_N_kl.py</code> ; <code>compare_N_sot.py</code>
L	<code>compare_L_cos.py</code> ; <code>compare_L_kl.py</code> ; <code>compare_L_sot.py</code>
Sr	<code>compare_Sr_cos.py</code> ; <code>compare_Sr_kl.py</code> ; <code>compare_Sr_sot.py</code>
$E + N$	<code>compare_E_N_cos.py</code> ; <code>compare_E_N_kl.py</code> ; <code>compare_E_N_sot.py</code>
$E + L$	<code>compare_E_L_cos.py</code> ; <code>compare_E_L_kl.py</code> ; <code>compare_E_L_sot.py</code>

$E + Sr$	<code>compare_E_Sr_cos.py</code> ; <code>compare_E_Sr_kl.py</code> ; <code>compare_E_Sr_sot.py</code>
$N + L$	<code>compare_N_L_cos.py</code> ; <code>compare_N_L_kl.py</code> ; <code>compare_N_L_sot.py</code>
$N + Sr$	<code>compare_N_Sr_cos.py</code> ; <code>compare_N_Sr_kl.py</code> ; <code>compare_N_Sr_sot.py</code>
$L + Sr$	<code>compare_L_Sr_cos.py</code> ; <code>compare_L_Sr_kl.py</code> ; <code>compare_L_Sr_sot.py</code>

这些文件可以共享公共工具函数，例如特征读取、标准化、余弦距离、KL cost、稀疏候选集合、sparse Sinkhorn 等。但是共享工具函数不能替代具体格子文件。换言之，公共函数只负责复用计算逻辑；`compare_<feature>_<method>.py` 负责清楚声明这个格子到底使用哪一组特征、哪一种 PIJ 生成方式，以及它导出的 method metadata。

已有的 `mignet_ce/pij/registry.py` 可以只增加注册行；它是既有全局入口，不需要改名为 `compare_registry.py`。但是本组新增的具体 PIJ method 文件本身应保持 `compare_` 前缀，并且一个文件只对应矩阵里的一个格子。

2 横轴：LightCCI 下的特征构造

2.1 共同节点空间

对一个样本组仍记为

$$g = (c', t, \ell).$$

这里， g 表示一个样本组； c' 表示器官或组织； t 表示发育时间点； ℓ 表示层级。对该样本组，LightCCI 给出节点集合

$$V_{\ell,t}^{(c')} = \{\text{该器官、该时间点、该层级的所有节点}\}.$$

这里， $V_{\ell,t}^{(c')}$ 是当前层级的节点集合。如果 ℓ 是 cell/spot 层，节点是 cell 或 spot；如果 ℓ 是 domain 层，节点是 domain；如果 ℓ 是 gene 层，节点是 gene。

对一个时间对 (t_0, t_1) ，定义 source 节点集合和 target 节点集合：

$$S = V_{\ell,t_0}^{(c')}, \quad T = V_{\ell,t_1}^{(c')}.$$

这里， S 表示 source 时间点 t_0 的节点集合； T 表示 target 时间点 t_1 的节点集合。对任意 $i \in S$ 和 $j \in T$ ，后续所有特征都必须在这两个节点空间上对齐。

2.2 四类基础特征

本矩阵的基础特征分为四类：表达特征、节点因子特征、Laplacian-HKS 边结构特征、SR 发育特征。为了避免符号混乱，记四类特征索引集合为

$$\mathcal{A}_0 = \{E, N, L, Sr\}.$$

这里， E 表示 expression feature； N 表示 node-factor feature； L 表示 Laplacian-HKS feature； Sr 表示 SR/developmental feature。使用 Sr 而不是 S 作为特征索引，是为了避免和 source 节点集合 S 混淆。

表达特征 x_i^E . 对节点 i , 表达特征记为

$$x_i^E \in \mathbb{R}^{d_E}.$$

这里, x_i^E 是节点 i 的表达向量; d_E 是表达特征维度。若节点是 **cell/spot/domain**, x_i^E 可以由该节点内部或该 **domain** 内细胞的基因表达聚合得到; 若节点是 **gene**, 则 x_i^E 应理解为该 **gene** 在 **cell/spot/domain** 维度上的表达轮廓, 而不是 **cell** 的 **gene-expression vector**。

表达特征列本身不需要使用 **LightCCI** 的 **CCI** 边。**LightCCI** 在这里提供节点集合、层级关系和对齐空间; 表达特征只从表达矩阵中读取并对齐到这些节点。

节点因子特征 x_i^N . 对节点 i , 节点因子特征记为

$$x_i^N \in \mathbb{R}^{d_N}.$$

这里, x_i^N 是通过 **joint NMF** 或同类非负因子分解得到的节点低维因子; d_N 是节点因子维度。

一个抽象写法是: 对每个时间点构造非负节点关系矩阵

$$X_t^N \in \mathbb{R}_+^{|V_{\ell,t}^{(c')}| \times p_N}.$$

这里, X_t^N 的每一行对应一个节点; p_N 是关系上下文维度, 可以来自节点对 **LR/CCI** 模式、邻居通讯画像、或其他非负节点关系描述。**joint NMF** 令

$$X_t^N \approx W_t H.$$

这里, $W_t \in \mathbb{R}_+^{|V_{\ell,t}^{(c')}| \times d_N}$ 是时间点 t 的节点因子矩阵; $H \in \mathbb{R}_+^{d_N \times p_N}$ 是跨时间共享的基矩阵。节点 i 的节点因子特征 x_i^N 就是 W_t 中与节点 i 对应的那一行。

本版最终口径: 邻接矩阵版 Joint NMF. 为了尊重来源文件中已有的 **joint NMF** 过程, 本版将节点因子特征 N 明确定义为直接接收 **LightCCI** 的邻接矩阵, 而不是额外改造成 **LR-pair profile** 或其他新型通讯通道画像。对一个样本组 $g = (c', t, \ell)$, **LightCCI** 给出非负邻接矩阵

$$A_{\ell,t}^{(c')} \in \mathbb{R}_+^{n_t \times n_t}.$$

这里, $A_{\ell,t}^{(c')}$ 是该器官、该时间点、该层级的 **LightCCI** 邻接矩阵; $n_t = |V_{\ell,t}^{(c')}|$ 是该时间点该层级的节点数; 矩阵元素 $A_{\ell,t}^{(c')}(u, v)$ 表示节点 u 到节点 v 的通讯、调控或聚合边强度, 具体边类型由层级决定: **cell/spot** 层主要来自 **CCI**, **domain** 层来自 **DDI** 或聚合 **CCI/DDI**, **gene** 层来自 **gene×gene GRN**。

在本矩阵中, 节点因子特征列 N 的输入矩阵定义为

$$X_t^N = A_{\ell,t}^{(c')}.$$

这里, X_t^N 是送入 **joint NMF** 的非负矩阵; 它的每一行对应一个 **source/target** 时间点内部的 **LightCCI** 节点; 它的每一列对应该节点指向其他节点的邻接模式。因此, X_t^N 保留的是节点的 **outgoing connectivity pattern**。第一版不额外拼接 $A_{\ell,t}^{(c')\top}$, 也不额外引入 **LR-pair profile**, 避免偏离原来源代码中“对输入矩阵做 **temporal joint NMF**”的主流程。

对多个时间点的邻接矩阵列表 $\{X_t^N\}_t$, joint NMF 仍然使用源代码中的同一类目标函数:

$$\min_{\{W_t^N\}, H^N \geq 0} \sum_t \|X_t^N - W_t^N H^N\|_F^2.$$

这里, $W_t^N \in \mathbb{R}_+^{n_t \times d_N}$ 是时间点 t 的节点因子矩阵; $H^N \in \mathbb{R}_+^{d_N \times p_N}$ 是跨时间共享的非负基矩阵; d_N 是 joint NMF 的低维因子数; p_N 是输入矩阵的列数。节点 i 的节点因子特征定义为

$$x_i^N = W_t^N(i, :).$$

这里, $W_t^N(i, :)$ 表示矩阵 W_t^N 中与节点 i 对应的那一行。这个向量可以解释为: 节点 i 在若干个 latent adjacency/connectivity pattern 上的负载。

需要保留一个工程边界: 源代码中的 joint NMF 会要求所有输入矩阵拥有相同列数。因此, 若直接令 $X_t^N = A_{\ell, t}^{(c')}$, 则同一批 joint NMF 输入需要满足

$$p_N = n_{t_1} = n_{t_2} = \dots.$$

这里, p_N 是 joint NMF 的共享列维度; $n_{t_1}, n_{t_2}, \dots$ 是不同时间点的节点数。第一版不做隐式 padding、不做截断, 也不把 adjacency 强行改写成 LR-pair profile; 如果列数不一致, 相关 compare_N_*.py 或包含 N 的格子文件应输出清晰的 shape diagnostic, 并在结果 metadata 中记录该格子不可运行的原因。

因此, 本版 N 的最终解释是: 基于 LightCCI 邻接矩阵的 joint NMF 节点因子特征。它和 Laplacian-HKS 同样来自 LightCCI 图, 但二者的表征机制不同: Laplacian-HKS 是

$$A_{\ell, t}^{(c')} \rightarrow L_{\ell, t} \rightarrow z_i^{lap},$$

即由边结构诱导的谱扩散节点特征; Joint NMF 是

$$A_{\ell, t}^{(c')} \rightarrow W_t^N \rightarrow x_i^N,$$

即由邻接模式低秩分解得到的节点因子特征。

Laplacian-HKS 特征 x_i^L . 对节点 i , Laplacian-HKS 特征记为

$$x_i^L = z_i^{lap} \in \mathbb{R}^{d_L}.$$

这里, z_i^{lap} 是前文已经定义的 HKS 节点结构表征; d_L 是 HKS 尺度数量或保留维度。它使用 LightCCI 图的边结构, 因此属于基于网络边的结构特征。

SR 发育特征 x_i^{Sr} . 对节点 i , SR 发育特征记为

$$x_i^{Sr} = q_i \in \mathbb{R}^{d_{Sr}}.$$

这里, q_i 表示节点 i 的 SR 或发育状态量; d_{Sr} 是 SR 特征维度。若 SR 是标量, 则 $d_{Sr} = 1$; 若 SR 被扩展成多个发育统计量, 则 $d_{Sr} > 1$ 。

2.3 组合特征

对任意非空特征子集

$$\mathcal{A} \subseteq \mathcal{A}_0,$$

定义组合特征

$$f_i^{\mathcal{A}} = \text{concat}_{a \in \mathcal{A}} (\sqrt{\omega_a} \hat{x}_i^a).$$

这里, $f_i^{\mathcal{A}}$ 表示节点 i 在特征组合 $\mathcal{A}$ 下的最终特征; a 表示某一类基础特征; $\omega_a \geq 0$ 是第 a 类特征的权重; $\hat{x}_i^a$ 表示经过标准化后的第 a 类特征; **concat** 表示向量拼接。

标准化的目的是避免表达特征维度大、SR 维度小而导致某一类特征天然支配距离。第一版可以对每一类基础特征分别做全局 **z-score** 或 **min-max scaling**, 再拼接。

当前矩阵先包含四个单特征列和六个两两组合列:

$$\begin{aligned} &\{E\}, \{N\}, \{L\}, \{Sr\}, \\ &\{E, N\}, \{E, L\}, \{E, Sr\}, \{N, L\}, \{N, Sr\}, \{L, Sr\}. \end{aligned}$$

这里暂不把三特征组合和四特征组合放进第一版完整矩阵。这样矩阵规模是 $10 \times 3 = 30$ 个核心实验, 并且对应上文 30 个 **compare_PIJ** 文件。

3 纵轴：从特征到 PIJ 的三类方法

对任意特征组合 $\mathcal{A}$, **source** 节点 $i \in S$ 和 **target** 节点 $j \in T$ 的基础距离统一使用余弦距离:

$$d_{\mathcal{A}}(i, j) = 1 - \frac{\langle f_i^{\mathcal{A}}, f_j^{\mathcal{A}} \rangle}{\|f_i^{\mathcal{A}}\|_2 \|f_j^{\mathcal{A}}\|_2 + \delta}.$$

这里, $d_{\mathcal{A}}(i, j)$ 是特征组合 $\mathcal{A}$ 下的 **source-target** 距离; $f_i^{\mathcal{A}}$ 是 **source** 节点 i 的组合特征; $f_j^{\mathcal{A}}$ 是 **target** 节点 j 的组合特征; $\langle \cdot, \cdot \rangle$ 表示内积; $\|\cdot\|_2$ 表示二范数; $\delta > 0$ 是数值稳定项。

3.1 Cosine similarity 到 PIJ

第一类 PIJ 方法直接由余弦距离生成转移核。定义

$$K_{ij}^{\mathcal{A}} = \exp\left(-\frac{d_{\mathcal{A}}(i, j)}{\tau}\right).$$

这里, $K_{ij}^{\mathcal{A}}$ 是从 **source** 节点 i 到 **target** 节点 j 的非负相似度核; $\tau > 0$ 是温度参数。随后按 **source** 行归一化:

$$P_{ij}^{\mathcal{A}, \text{cos}} = \frac{K_{ij}^{\mathcal{A}}}{\sum_{j' \in T} K_{ij'}^{\mathcal{A}} + \delta}.$$

这里, $P_{ij}^{\mathcal{A}, \text{cos}}$ 是 cosine similarity 方法得到的 PIJ; j' 表示 **target** 节点索引; δ 是数值稳定项。

3.2 Feature-level KL 到 PIJ

第二类方法先把组合特征转成概率分布，再用 feature-level KL 作为 source-target cost。定义

$$\pi_i^{\mathcal{A}} = \text{softmax}\left(\frac{f_i^{\mathcal{A}}}{\beta}\right), \quad \pi_j'^{\mathcal{A}} = \text{softmax}\left(\frac{f_j'^{\mathcal{A}}}{\beta}\right).$$

这里， $\pi_i^{\mathcal{A}}$ 是 source 节点 i 的特征分布； $\pi_j'^{\mathcal{A}}$ 是 target 节点 j 的特征分布； $\beta > 0$ 是 softmax 温度。

feature-level KL cost 定义为

$$D_{ij}^{\mathcal{A}, KL} = \text{KL}(\pi_i^{\mathcal{A}} \parallel \pi_j'^{\mathcal{A}}) = \sum_k \pi_{i,k}^{\mathcal{A}} \log \frac{\pi_{i,k}^{\mathcal{A}} + \delta}{\pi_{j,k}'^{\mathcal{A}} + \delta}.$$

这里， $D_{ij}^{\mathcal{A}, KL}$ 是 source 节点 i 到 target 节点 j 的 feature-level KL cost； k 是特征维度索引； δ 是数值稳定项。

再定义 KL 核并行归一化：

$$K_{ij}^{\mathcal{A}, KL} = \exp\left(-\frac{D_{ij}^{\mathcal{A}, KL}}{\tau_{KL}}\right),$$

$$P_{ij}^{\mathcal{A}, KL} = \frac{K_{ij}^{\mathcal{A}, KL}}{\sum_{j' \in T} K_{ij'}^{\mathcal{A}, KL} + \delta}.$$

这里， $\tau_{KL} > 0$ 是 KL kernel 的温度参数。需要注意， $D_{ij}^{\mathcal{A}, KL}$ 是特征分布之间的 KL；它不同于 semi-relaxed OT 目标函数中约束 source 质量的 KL。

3.3 Sparse Semi-relaxed OT 到 PIJ

第三类方法使用 sparse semi-relaxed OT。和前文综合主方案不同，本矩阵里的 OT 版本不再把 Laplacian、SR、空间坐标分别写成多个预成本分量，也不加入辅助图内部边。对每一个特征组合 $\mathcal{A}$ ，预成本只定义为该组合特征的余弦距离：

$$B_{ij}^{\mathcal{A}} = d_{\mathcal{A}}(i, j).$$

这里， $B_{ij}^{\mathcal{A}}$ 是矩阵版 OT 的 source-target 预成本；它只来自当前特征组合 $\mathcal{A}$ 的距离。若 $\mathcal{A} = \{L\}$ ，它只使用 Laplacian-HKS 距离；若 $\mathcal{A} = \{L, Sr\}$ ，它使用拼接后的 Laplacian-HKS 与 SR 组合特征距离；不会额外再把空间距离单独加进去。

候选集合仍由小预成本筛选：

$$\Omega = \Omega_{S \rightarrow T} \cup \Omega_{T \leftarrow S},$$

$$\Omega_{S \rightarrow T} = \{(i, j) : j \in \text{TopKsmall}_{K_s}(B_{i,:}^{\mathcal{A}})\},$$

$$\Omega_{T \leftarrow S} = \{(i, j) : i \in \text{TopKsmall}_{K_t}(B_{:,j}^{\mathcal{A}})\}.$$

这里， Ω 是稀疏跨时间候选边集合； K_s 是每个 source 节点保留的 target 候选数； K_t 是每个 target 节点反向保留的 source 候选数。

最终 OT cost 为

$$C_{ij}^A = \text{normalize}_{(i,j) \in \Omega} (B_{ij}^A), \quad (i, j) \in \Omega.$$

这里, C_{ij}^A 是进入 OT 的稀疏成本; **normalize** 表示只在候选边集合 Ω 上做尺度归一化, 例如分位数截断后缩放到 $[0, 1]$ 。

在候选集合上求解

$$P \in \mathbb{R}_+^{|S| \times |T|}, \quad \text{supp}(P) \subseteq \Omega.$$

这里, P_{ij} 表示 source 节点 i 到 target 节点 j 的转移强度; $\text{supp}(P)$ 表示 P 的非零位置集合。

矩阵版 semi-relaxed OT 目标为

$$\begin{aligned} \min_{P \geq 0, \text{supp}(P) \subseteq \Omega} \quad & \sum_{(i,j) \in \Omega} P_{ij} C_{ij}^A + \varepsilon \sum_{(i,j) \in \Omega} P_{ij} (\log P_{ij} - 1) + \gamma \text{KL}(\mathbf{P}\mathbf{1} \| a), \\ \text{s.t.} \quad & P^\top \mathbf{1} = b. \end{aligned}$$

这里, 第一项让较大的转移质量尽量落在较小的特征距离上; 第二项是 **entropy regularization**; $\varepsilon > 0$ 控制平滑强度; 第三项是 source 侧质量守恒的软约束; $\gamma > 0$ 控制 source 质量偏离的惩罚强度; a 是 source 节点质量; b 是 target 节点质量; $P^\top \mathbf{1} = b$ 表示 target 侧质量使用硬约束。

该方法导出两个版本: 未归一化的 raw transport P , 以及供 EI 计算使用的行归一化矩阵

$$\hat{P}_{ij} = \frac{P_{ij}}{\sum_{j' \in T} P_{ij'} + \delta}.$$

这里, $\hat{P}_{ij}$ 是行归一化后的 PIJ; raw transport P 用于 source mass 诊断。

4 完整实验矩阵

核心矩阵如下: 横轴是特征组合, 纵轴是 PIJ 生成方法。该表的每一个勾选格子都对应一个独立 **compare_PIJ** 文件。

PIJ 方法 / 特征	E	N	L	Sr	$E + N$	$E + L$	$E + Sr$	$N + L$	$N + Sr$	$L + Sr$
Cosine kernel	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Feature-level KL	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Sparse semi-relaxed OT	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

其中, E 是表达特征; N 是节点因子特征; L 是 Laplacian-HKS 特征; Sr 是 SR 发育特征。该表对应 30 个核心实验, 也对应 30 个独立 PIJ 方法文件。

5 综合主方案与矩阵版 OT 的区别

前文原始综合方案可以保留为一个单独版本, 例如称为 “LightCCI 综合版”。该版本的特点是: Laplacian-HKS、SR、空间坐标共同构造预成本, 并且曾讨论过辅助图内部边或路径成本。它适合作为主线方法或增强版方法, 但不适合直接放进本二维消融矩阵里, 因为它已经同时混合了多类 feature 和额外图结构机制。

因此，本矩阵中的 sparse semi-relaxed OT 应定义为更干净版本：

$$\text{当前特征组合 } \mathcal{A} \rightarrow d_{\mathcal{A}}(i, j) \rightarrow B_{ij}^{\mathcal{A}} \rightarrow \Omega \rightarrow C_{ij}^{\mathcal{A}} \rightarrow P_{ij}.$$

这里， $d_{\mathcal{A}}(i, j)$ 统一使用余弦距离； $B_{ij}^{\mathcal{A}}$ 只来自当前特征组合； Ω 只由该预成本筛出； $C_{ij}^{\mathcal{A}}$ 只是在候选边上的归一化特征成本。空间坐标、辅助图内部边和多分量加权预成本不进入这组矩阵版 OT。

6 实现边界总结

本组对比实验应满足以下边界：

1. LightCCI 是共同的网络层级构造器；它提供节点、层级图和对齐空间。
2. 特征列决定是否使用 CCI 边：表达特征列可以完全不使用 CCI 边；Laplacian-HKS 列使用 CCI/DDI/GRN 边；节点因子列是否使用 CCI 取决于节点关系矩阵如何定义。
3. 新增 PIJ 代码文件统一放在 `mignet_ce/pij/` 下，并使用 `compare_` 前缀。
4. 矩阵里的每个格子必须是一个独立 PIJ 文件；不能只用 `compare_cos.py`、`compare_kl.py` 或 `compare_sot.py` 通过参数切换不同特征列。
5. 矩阵版 sparse semi-relaxed OT 的预成本只等于当前特征组合的余弦距离，不再默认混入 SR、空间和辅助图成本。
6. 原始综合方案可以作为单独增强版本保留，但不算入 30 个核心矩阵实验。